M2: Mid-Submission

Team Number: **191**

Problem Statement: Automatic Category Suggestions for Wikimedia Commons Images

Current approach and changes since M1

Our pipeline converts each image into a 512-dimensional CLIP (ViT-B/32) embedding and ranks categories with four complementary scorers: (1) zero-shot matching of the image against the category name, (2) k-nearest-neighbour voting over known images, (3) a small trained linear head with a per-category sigmoid and asymmetric loss, and (4) a title/description scorer using TF-IDF neighbour voting. A logistic-regression meta-classifier fuses the four scores of each candidate category; it is fitted on the validation split only, with one set of global weights learned across all categories. The Commons category graph is then used to drop a suggested category when a more specific descendant is also in the list, which matches the editors' most-specific-category rule. The demo is a Gradio page where a human uploads an image (and optional title), sees ranked suggestions with the six most similar known images as the explanation, and accepts or rejects each suggestion.

Since M1 we have made four concrete changes. First, we added the title/description scorer, because Commons filenames often name the subject. Second, we fetch the real category graph from the MediaWiki API two levels up (22,413 edges) instead of relying only on parents present in our sample; only 8 ancestor links fall inside our current vocabulary, so the effect on our sample is small. Third, we use a strict time-based split (oldest 70% train, next 15% validation, newest 15% test) and remove validation/test images whose cosine similarity to any training image is at least 0.97. Fourth, we cleaned the labels by filtering process categories and, after reading the most frequent categories, added `"photos by"` to the junk list because a photographer's category appeared as a suggestion.

We have also identified a coverage gap: the problem statement lists objects, scenes, events and abstract concepts, but our current 23 working seed categories are mostly objects, places and scenes. Eiffel Tower has no plain category in the vocabulary (its images sit under replicas and art categories), Steam locomotives was dropped because its images are spread over many subcategories, and there is no abstract category. We have updated `download_data.py` to add five new event categories (Durga Puja, Oktoberfest, Fireworks, Carnivals, Marathons), six abstract/pattern categories and eight small categories of about 40 images each, to replace Steam locomotives, to take images from many subcategories so one photographer's batch cannot dominate, and to verify category names before the long download. The new names still need to be verified and the download run. We will lower `N_MIN` to 5 and aim for 3,000+ images in total.

Experiments run so far and results

The current run uses 2,096 downloaded images. After junk-category filtering and keeping categories with at least 8 training images, 52 categories remain, with about 1.2 labels per image (train 1,408 / val 308 / test 307). Table 1 shows results on the held-out *test* split, which was never used for fitting. The fused model beats every single scorer. Precision@5 is only 0.201 because an image has about 1.2 true labels, so P@5 cannot exceed roughly 0.24 whatever the model does; we therefore lead with P@1, R@5 and mAP.

Scorer	P@1
Zero-shot	0.746
Title/description	0.765
kNN	0.821
Trained head	0.860
Fusion	**0.938**

Table 1: Test results (strict labels, time split, 307 images, 52 categories). The fused model also reaches R@5 0.982 and mAP 0.958.

Labels are incomplete, so strict scores undercount true quality. With the category graph, strict P@5 is 0.201, lenient P@5 (ancestors of true labels accepted) is 0.207, and after ancestor suppression P@5 is 0.198 with an average list length of 4.78. Hard mode (seed label hidden) and the head/torso/tail breakdown are computed by `evaluate.py`, but only 5 and 10 test images respectively have the relevant labels, so those numbers are noise and we do not report them; the larger sample is meant to fix this.

Robustness (fused, test images damaged, measured before the `"photos by"` fix): R@5 is 0.982 clean, 0.973 under blur, 0.982 under downscale-then-upscale and 0.975 under JPEG quality 10. Tiny images change the embeddings the most (mean cosine to the clean embedding 0.70), and the image-only scorers lose 7–13 points of P@1 (kNN 0.821→0.746, head 0.857→0.723, zero-shot 0.720→0.632). The title/description scorer is unaffected (0.765), and fusion with it absorbs most of the loss; with camera-number filenames in real uploads, this help

would be smaller. Latency on CPU only is a median of 132.1 ms and p95 of 140.7 ms per suggestion (about 7 images per second for one process), and the index of 2,024 images is 4.1 MB. At scale, 100M images with product quantization (64 bytes per vector) would be about 6.4 GB with approximate search in milliseconds.

Progress on testing

We have completed the time-based split, near-duplicate removal, label cleaning and an evaluation script that produces P@k, R@k, F1@k, mAP and the tier breakdowns. The robustness and latency scripts work. The Gradio demo accepts an upload, shows the top suggestions with scores and the six most similar known images, and logs accept/reject feedback; accepted images join the kNN index immediately, while the head and text scorer wait for the next retrain. In demo tests a Statue of Liberty photo gets Statue of Liberty as the top suggestion with six matching gallery images, and a laptop photo (outside our categories) gets an empty list. Only one outside image has been tested, and the 0.05 display cutoff is an untuned first guess.

We are adding the following tests, which address the remaining gaps. A results-by-category-type table will report objects, scenes, events and abstract concepts separately. An out-of-category cutoff test will take 15–20 outside images, choose the score cutoff on the validation split and replace the untuned 0.05. A throughput and resolution-band table will report images per second and results by image size. A retrain policy script will fold accepted feedback images into `train.py` and log the acceptance rate. A language test will measure how the text scorer behaves on non-English titles, so the inclusivity point has a real number. If time allows, we will also report a reviewed score on 50 test images next to the strict one.

Plan for remaining time

1. **Larger, broader data:** verify the new category names with `download_data.py check`, fix any wrong ones, run the full re-download (events, abstract concepts, small categories), then `fetch_graph.py`.
2. **New evaluation code (written while download and embedding run):** results by category type, out-of-category cutoff test, throughput and resolution bands, retrain policy script, and the non-English title test.
3. **Re-run on the expanded data:** `embed_images.py`, `prepare_labels.py` with `N_MIN=5`, `train.py` and `evaluate.py`. All numbers above will be re-measured, and hard mode and head/torso/tail will be reported only if enough test images exist.
4. **Demo:** complete the feedback loop with the retrain policy and re-check the demo on the new data.
5. **Final deliverables:** push the full code to `main` with a README (setup and run instructions only) and an MIT LICENSE, write the 3-page `deliverables/final_report.pdf`, record the demo video (at most 4 minutes, with voiceover) and share it through Google Drive with "Anyone with the link" access, and zip the repository without virtual environment, datasets or model weights. Slides follow at 11:30 AM.
6. **Optional:** reviewed score on 50 test images, reported next to the strict score.

The main risk is re-download time. We are starting it immediately and keeping the laptop awake. If it fails, we will fall back to the current dataset and state the coverage limitation clearly. Our results apply to our own sample, not to all of Commons.